Rotaract Club of Xiamen University Malaysia

Rotary Club of Kuala Lumpur Dynamics

Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: [Insert date]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	ProjectKita
Project / Solution Name	Payment Follow-up
Name (Leader)	Muhammad Fikri Bin Mohd Nizan
Email & Phone (Leader)	fikrinizan04@gmail.com (+60-19 886 7107)
University Affiliation(s)	Asia Pacific University of Technology and Innovation (APU)
Live Prototype URL	https://payment-follow-up.replit.app
Code Repository URL (public / access granted to judges)	https://github.com/XJ-Ong/Payment-Follow-up

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

User access a react application running on Vite. Frontend sends API requests to FastAPI backend with simulated role-based access control in demo mode. Backend contains three main layers, which are API routes, service layer (business logic), PostgreSQL database.

Scoring engine uses deterministic calculation for priority scores (outstanding x overdue days) to identify which invoice require follow-up. When reminder is needed, the reminder message is drafted by an AI service connected to Runpod (Ollama). AI outputs automatically fallback to templates if they fail. APScheduler handles optional automated daily scoring recalculation.

Orbyte adapter layer uses a Protocol interface with mock implementation reading JSON seed files. It is designed to swap in real Orbyte API calls in real production. State changes are logged to an append-only audit trail. System simulates sending reminders through WhatsApp, Email, and Telegram channels.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	**Purpose in Project**	**License**
Python 3.12+	Backend language	PSF License
FastAPI 0.159.1+	Backend web framework and API	MIT
PostgreSQL (latest)	Relational database	PostgreSQL
SQLAlchemy 2.1.6+	Database ORM and query builder	MIT
Alembic 1.13+	Database migration management	MIT

Pydantic 2.14.1+	Data validation and settings	MIT
APScheduler 3.11.3+	Background job scheduling	MIT
Uvicorn 0.54.0+	ASGI web server	BSD-3
psycopg2-binary 2.9.13+	PostgreSQL adapter for Python	LGPL
OpenAI Python SDK 1.0+	AI model client library	MIT
pytest 8+	Testing framework	MIT
httpx 0.27+	HTTP client for API testing	BSD-3
python-dotenv 1.2.4+	Environment variable management	BSD-3
Node.js 18+	Frontend build tooling runtime	MIT
React 19.1.0	Frontend UI framework	MIT
Vite 6.4.5	Frontend build tool and dev server	MIT
TypeScript 5.8.3	Type-safe JavaScript	Apache 2.0
Tailwind CSS 4.1.0	Utility-first CSS framework	MIT
Recharts 3.0.0	React charting library	MIT

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
Runpod Serverless API	AI model inference for reminder drafting	Paid (serverless compute billing)
Telegram Bot API	Simulated reminder delivery channel (partial)	Free

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Runpod Serverless	Hosting open-source LLM with vLLM	Serverless
GitHub	Source control and collaboration	Free
Replit	Application Deployment	Core

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Qwen2.5-Coder 1.5B	Alibaba Cloud	Pre-trained (hosted on Runpod via vLLM)	Apache 2.0

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Synthetic Demo Data	Self-created	Synthetic	N/A	N

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
opencode	Executing Code Implementation Plan	major
Claude Code	Code and Security review	minor
Claude web client	Consult and learning material	moderate
Github Copilot	Handle merge conflict	moderate
Codex	Design system plan	major

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

AI tools drafted code and plans, but humans owned every decision. Implementation plans were written and approved by the team lead before coding. All AI-generated code went through personal branches and pull requests into a shared dev branch, where teammates reviewed it before merging to main. Core logic, such as priority scoring, is covered by automated tests using hand-calculated examples. A separate security review checked the code. At runtime, the product itself treats AI output as untrusted: numbers are verified against source data, and a person approves every message. We take full responsibility for the submitted work.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	Hosting platform	The prototype is deployed on a Replit Reserved VM with its own PostgreSQL. The workshop shaped our decision to use Replit for deployment and hosting only, because credit limits could run out mid-development
Ronpod	GPU Pod	A Runpod GPU Pod serves our open-source language model, which drafts reminders, explains rankings and writes the weekly summary.
Alpha Red (Orbyte)	Domain briefing and Orbyte lab environment	Exploring the lab environment shaped our assumed Orbyte data design (clients, projects, users), which we simulate through an adapter layer.

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Not essential to the core logic, and replaceable. Scoring, alerts, approvals, roles and the activity log run in our own backend and database. The AI only writes and explains, and it sits behind one service with a fixed-template fallback, so the journey still completes if the model is unavailable. Runpod could be swapped for another GPU host or model provider, and Replit for any hosting service. Both platforms made the prototype faster to deploy and let us keep customer data away from third-party AI APIs, but neither locks us in.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The prototype uses only synthetic demonstration data. No real personal or business data was collected. In a production deployment, the system would collect customer names, contact preferences (email/WhatsApp/phone), invoice amounts, due dates, and payment history. Role-based access control limits visibility to Owner and Finance roles only.

When AI drafting is enabled, only the minimum necessary fields (customer name, invoice number, amount, due date, days overdue, preferred channel) are sent to the AI endpoint. No contact details (phone/email) are transmitted. The team's architecture uses a self-hosted AI model on Runpod Serverless to maintain data privacy control.

All state changes are logged in an append-only audit trail. Database credentials and API keys are stored in environment variables, never committed to source control. Demo mode (X-Role header simulation) is explicitly disabled for any public deployment per security documentation.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

AI Hallucination: The model may generate incorrect amounts or dates. Mitigation: mandatory number verification checks every AI-generated message against source data; mismatches trigger automatic fallback to template-based messages.

Template Fallback: Smaller models (1.5B parameters) may produce lower-quality Bahasa Malaysia text. Templates ensure the system completes its journey even with AI unavailable.

Rate Limits: Runpod Serverless endpoints have request limits. Mitigation: template fallback ensures functionality; production would implement request queuing.

Tone Control: AI may not perfectly match the requested tone (friendly/neutral/firm). Mitigation: human approval required before any message is sent; approvers can edit drafts.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.



(25/10/2026)